

ФГБОУ ВО «Пензенский государственный университет»

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по дисциплине «Логика и ОА»

лабораторная работа №3 на тему:

«Динамические списки»

Выполнил студенты группы 25ВВВ3:

Кустов И. С

Никитин К. А

Приняли:

к.т.н., доцент Юрова О. В.

к.т.н., доцент Деев М. В.

Тема: «Динамические списки»

Цель работы: Изучить динамические структуры данных на примере односвязного линейного списка. Освоить основные операции над списком: создание элемента, добавление в конец (и в соответствии с приоритетом), просмотр, поиск по заданному ключу и удаление. Научиться применять динамический список для реализации абстрактных структур данных: приоритетной очереди, обычной очереди и стека. Закрепить навыки работы с указателями, динамическим выделением памяти (`malloc`, `free`) и структурами (`struct`) в языке C.

Лабораторное задание:

Задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного кода реализуйте структуру данных Очередь.
3. На основе приведенного кода реализуйте структуру данных Стек.

Пояснительный текст:

1. Реализация приоритетной очереди, путём добавления элемента в список в соответствии с приоритетом объекта

Назначение программы

Программа реализует структуру данных приоритетная очередь с помощью односвязного динамического списка. В такой очереди элементы извлекаются не в порядке добавления (FIFO), а в соответствии с их приоритетом: элемент с наибольшим приоритетом всегда находится в начале списка и извлекается первым. Если у нескольких элементов одинаковый приоритет, они сохраняют порядок добавления относительно друг друга.

Структура данных

Каждый элемент списка описывается структурой `struct node`, которая содержит:

- `char inf[256]` - информационная часть (название объекта),

- `int priority` - приоритет (целое число; чем больше значение, тем выше приоритет),

- `struct node* next` - указатель на следующий элемент списка.

Глобальная переменная `head` хранит указатель на первый (головной) элемент списка. Изначально `head = NULL`, что означает пустую очередь.

Функция `push` - добавление элемента с учётом приоритета.

Это ключевая функция, обеспечивающая упорядоченность очереди по убыванию приоритета. Алгоритм работает следующим образом:

Выделяется память под новый узел `p` с помощью `malloc`.

В поля узла копируются название (`strcpy_s`) и приоритет, указатель `next` устанавливается в `NULL`.

Проверяется условие: если список пуст (`head == NULL`) или приоритет нового элемента больше приоритета головного (`pr > head->priority`), то новый элемент становится новым `head`:

```
p->next = head;
```

```
head = p;
```

Иначе (приоритет нового элемента не выше головного) выполняется поиск места вставки. Временный указатель `curr` начинает с головы. Цикл `while` продолжается, пока следующий элемент существует и его приоритет больше или равен приоритету нового элемента:

```
while (curr->next != NULL && curr->next->priority >= pr)
```

```
    curr = curr->next;
```

То есть пропускаем все элементы, приоритет которых не меньше нового. Как только встречаем элемент с приоритетом меньше нового (или доходим до конца), цикл останавливается.

Новый элемент вставляется после `curr`:

```
p->next = curr->next;
```

```
curr->next = p;
```

Таким образом, список всегда отсортирован по убыванию приоритета: в голове - самый приоритетный, в хвосте - наименее приоритетный.

Функция `pop` - извлечение элемента с наивысшим приоритетом.

Поскольку список упорядочен, элемент с максимальным приоритетом всегда находится в голове. Функция `pop`:

Проверяет, пуст ли список. Если да, выводит сообщение «Очередь пуста!» и возвращает управление.

Иначе сохраняет указатель на голову во временную переменную `temp`, выводит информацию об извлекаемом элементе (название и приоритет).

Перемещает голову на следующий элемент: `head = head->next`.

Освобождает память, занятую извлечённым узлом: `free(temp)`.

Таким образом, извлекается именно самый приоритетный элемент.

Функция `print_list` - просмотр очереди.

Проходит по всем элементам списка, начиная с головы, и выводит для каждого приоритет и название. Если список пуст, выводится соответствующее сообщение. Используется для наглядной демонстрации текущего состояния очереди.

Функция `main` - интерфейс пользователя.

Организует диалог с пользователем через меню:

- Добавить элемент - запрашивает название (строка) и приоритет (целое число), затем вызывает `push`.
- Извлечь элемент (`pop`) - вызывает `pop`, которая удаляет и выводит головной элемент.
- Показать очередь - вызывает `print_list` для отображения всех элементов.
- Выход - завершает программу.

Ввод осуществляется с использованием безопасных функций `scanf_s`, `gets_s`, что соответствует требованиям Microsoft и позволяет избежать переполнения буфера. Для корректного отображения кириллицы настроена кодовая страница консоли (`SetConsoleCP`, `SetConsoleOutputCP`).

Реализация приоритетной очереди:

```
1. Добавить элемент
2. Извлечь элемент (pop)
3. Показать очередь
0. Выход
Выбор: 1
Введите название объекта: Россия
Введите его приоритет (целое число): 100

1. Добавить элемент
2. Извлечь элемент (pop)
3. Показать очередь
0. Выход
Выбор: 1
Введите название объекта: СССР
Введите его приоритет (целое число): 90

1. Добавить элемент
2. Извлечь элемент (pop)
3. Показать очередь
0. Выход
Выбор: 1
Введите название объекта: Китай
Введите его приоритет (целое число): 75

1. Добавить элемент
2. Извлечь элемент (pop)
3. Показать очередь
0. Выход
Выбор: 3

--- ТЕКУЩАЯ ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
[Приоритет: 100] Россия
[Приоритет: 90] СССР
[Приоритет: 75] Китай
-----

1. Добавить элемент
2. Извлечь элемент (pop)
3. Показать очередь
0. Выход
Выбор: 2
Извлечен элемент: Россия (приоритет: 100)
```

2. Реализация структуры данных Очередь

Назначение программы

Программа реализует структуру данных очередь (FIFO - First In, First Out) на основе односвязного динамического списка. В очереди первым извлекается тот элемент, который был добавлен раньше всех. Это соответствует принципу «первый пришёл - первый вышел». Программа позволяет добавлять элементы в конец очереди, извлекать их из начала и просматривать текущее содержимое.

Структура данных

Каждый элемент очереди описывается структурой `struct node`:

- `char inf[256]` - информационная часть (строковое значение элемента),
- `struct node* next` - указатель на следующий элемент списка.

Глобальные переменные:

- `head` - указатель на начало очереди (первый элемент, который будет извлечён),
- `last` - указатель на конец очереди (последний добавленный элемент).

Изначально оба указателя равны `NULL`, что означает пустую очередь.

Функция `push` - добавление элемента в конец очереди.

Алгоритм:

Выделяется память под новый узел `p` с помощью `malloc`.

В поле `inf` копируется переданная строка с помощью `strcpy_s` (безопасная версия `strcpy`).

Указатель `p->next` устанавливается в `NULL`, так как новый элемент будет последним.

Если очередь пуста (`head == NULL`), то новый элемент становится и началом, и концом: `head = last = p`.

Иначе новый элемент присоединяется после текущего последнего: `last->next = p`, а затем `last` обновляется на новый элемент: `last = p`.

Таким образом, добавление всегда происходит в конец, а указатель `last` позволяет делать это за $O(1)$, без прохода по всему списку.

Функция `pop` - извлечение элемента из начала очереди.

Алгоритм:

Проверяется, пуста ли очередь. Если `head == NULL`, выводится сообщение «Очередь пуста!» и функция завершается.

Иначе сохраняется указатель на первый элемент во временную переменную `temp`.

Выводится информация об извлекаемом элементе: `printf("Извлечен из очереди: %s\n", temp->inf);`.

Указатель `head` перемещается на следующий элемент: `head = head->next`.

Если после этого `head` стал `NULL` (то есть очередь опустела), то и `last` обнуляется: `last = NULL`.

Память, занятая извлечённым узлом, освобождается: `free(temp)`.

Таким образом, извлекается всегда первый добавленный элемент, что соответствует дисциплине FIFO. Операция выполняется за $O(1)$.

Функция `print_queue` - просмотр очереди.

Проходит по всем элементам списка, начиная с `head`, и выводит их значения в формате [значение] -> ... -> `NULL`. Если очередь пуста, выводится сообщение «Очередь пуста.». Функция используется для наглядного отображения текущего состояния очереди.

Функция `main` - интерфейс пользователя.

Организует диалог с пользователем через меню:

Встать в очередь (`push`) - запрашивает строку и вызывает `push`.

Выйти из очереди (`pop`) - вызывает `pop`, которая удаляет и выводит первый элемент.

Показать очередь - вызывает `print_queue`.

Выход - завершает программу.

Ввод осуществляется с использованием безопасных функций `scanf_s` и `gets_s`, что предотвращает переполнение буфера.

Реализация структуры данных Очередь:

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: 1

Введите значение: 20

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: 1

Введите значение: 30

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: 1

Введите значение: 25

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: 3

Очередь (FIFO): [20] -> [30] -> [25] -> NULL

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: 2

Извлечен из очереди: 20

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: 3

Очередь (FIFO): [30] -> [25] -> NULL

```
1. Встать в очередь (push)
2. Выйти из очереди (pop)
3. Показать очередь
0. Выход
```

Выбор: |

3. Реализация структуры данных Стек:

Назначение программы

Программа реализует структуру данных стек (LIFO - Last In, First Out) на основе односвязного динамического списка. В стеке первым извлекается тот элемент, который был добавлен последним. Это соответствует принципу «последним пришёл - первым вышел». Программа позволяет добавлять элементы в вершину стека, извлекать их из вершины и просматривать текущее содержимое.

Структура данных

Каждый элемент стека описывается структурой `struct node`:

- `char inf[256]` - информационная часть (строковое значение элемента),
- `struct node* next` - указатель на следующий элемент списка.

Глобальная переменная `head` хранит указатель на вершину стека (последний добавленный элемент). Изначально `head = NULL`, что означает пустой стек. В отличие от очереди, здесь нет отдельного указателя на хвост, так как все операции выполняются только с вершиной.

Функция `push` - добавление элемента в вершину стека.

Алгоритм:

Выделяется память под новый узел `p` с помощью `malloc`.

В поле `inf` копируется переданная строка с помощью `strcpy_s`.

Указатель `p->next` устанавливается на текущую вершину стека: `p->next = head`.

Вершина стека перемещается на новый элемент: `head = p`.

Таким образом, новый элемент всегда вставляется в начало списка (в вершину стека). Операция выполняется за $O(1)$, так как не требует прохода по списку. После добавления именно этот элемент станет первым при извлечении.

Функция `pop` - извлечение элемента из вершины стека.

Алгоритм:

Проверяется, пуст ли стек. Если `head == NULL`, выводится сообщение «Стек пуст!» и функция завершается.

Иначе сохраняется указатель на вершину во временную переменную `temp`.

Выводится информация об извлекаемом элементе: `printf("Вытолкнут из стека: %s\n", temp->inf);`.

Вершина стека перемещается на следующий элемент: `head = head->next`.

Память, занятая извлечённым узлом, освобождается: `free(temp)`.

Таким образом, извлекается всегда последний добавленный элемент - это и есть дисциплина LIFO. Операция также выполняется за $O(1)$.

Функция `print_stack` - просмотр стека.

Проходит по всем элементам списка, начиная с вершины (`head`), и выводит их значения в виде таблицы, где вершина отображается сверху. Формат вывода наглядно показывает порядок элементов в стеке. Если стек пуст, выводится сообщение «Стек пуст.». Функция используется для демонстрации текущего состояния стека.

Функция `main` - интерфейс пользователя.

Организует диалог с пользователем через меню:

Положить в стек (`push`) - запрашивает строку и вызывает `push`.

Забрать из стека (`pop`) - вызывает `pop`, которая удаляет и выводит элемент с вершины.

Показать стек - вызывает `print_stack`.

Выход - завершает программу.

Ввод осуществляется с использованием безопасных функций `scanf_s` и `gets_s`, что предотвращает переполнение буфера. Для корректного отображения кириллицы настроена кодовая страница консоли (`SetConsoleCP`, `SetConsoleOutputCP`).

Реализация структуры данных Стек:

```
1. Положить в стек (push)
2. Забрать из стека (pop)
3. Показать стек
0. Выход
Выбор: 1
Введите значение: 100

1. Положить в стек (push)
2. Забрать из стека (pop)
3. Показать стек
0. Выход
Выбор: 1
Введите значение: 50

1. Положить в стек (push)
2. Забрать из стека (pop)
3. Показать стек
0. Выход
Выбор: 1
Введите значение: 300

1. Положить в стек (push)
2. Забрать из стека (pop)
3. Показать стек
0. Выход
Выбор: 3

--- СОДЕРЖИМОЕ СТЕКА (ВЕРШИНА СВЕРХУ) ---
| 300                                     |
| 50                                     |
| 100                                    |
-----

1. Положить в стек (push)
2. Забрать из стека (pop)
3. Показать стек
0. Выход
Выбор: 2
Вытолкнут из стека: 300

1. Положить в стек (push)
2. Забрать из стека (pop)
3. Показать стек
0. Выход
Выбор: 3

--- СОДЕРЖИМОЕ СТЕКА (ВЕРШИНА СВЕРХУ) ---
| 50                                     |
| 100                                    |
-----
```

Вывод:

В ходе выполнения лабораторной работы были изучены принципы организации динамических списков и особенности работы с ними. Реализованы функции создания нового элемента, добавления его в список (в конец, а также с учётом приоритета), просмотра содержимого, поиска и удаления элемента по заданному значению. На основе базового односвязного списка была построена приоритетная очередь: при добавлении нового объекта он размещается в списке так, чтобы элементы с большим приоритетом находились перед элементами с меньшим.